

三、软件测试类选题相关问题

(1) 请简要介绍被测系统的整体架构和功能模块

整体架构：

- 前端：Vue 3 + TypeScript + Element Plus + Vite
- 后端：Spring Boot + MyBatis Plus + MySQL + Redis
- 安全：JWT + Spring Security

功能模块：

1. 用户管理：注册、登录、个人信息管理、地址管理
2. 商品管理：商品展示、分类管理、商品详情、评价系统
3. 订单管理：购物车、订单创建、订单状态管理、支付
4. 商家管理：商品发布、库存管理、订单处理
5. 管理员管理：用户管理、商品审核、商家审核、数据统计
6. 租赁系统：租赁商品管理、押金管理、租期管理
7. 二手交易：二手商品发布、成色评估、交易管理

(2) 你是如何制定测试计划的？能否分享一下你的测试计划的实施及关键内容？

测试计划制定：

1. 测试目标：验证系统功能是否符合需求，性能是否满足要求，安全性是否可靠
2. 测试范围：覆盖系统的所有功能模块，包括前端和后端
3. 测试类型：功能测试、性能测试、安全测试、兼容性测试
4. 测试方法：黑盒测试、白盒测试、集成测试、系统测试
5. 测试环境：开发环境、测试环境、生产环境
6. 测试工具：Postman、Jmeter、Selenium、SonarQube, APIFOX
7. 测试进度：根据开发进度制定测试计划，分阶段进行测试

测试计划实施：

1. 单元测试：对核心功能模块进行单元测试，确保功能正确性
2. 集成测试：测试不同模块之间的交互，确保系统整体功能正常
3. 系统测试：对整个系统进行测试，验证系统是否符合需求
4. 性能测试：测试系统在高并发场景下的性能表现
5. 安全测试：测试系统的安全性，包括漏洞扫描和渗透测试
6. 兼容性测试：测试系统在不同浏览器和设备上的表现

关键内容：

- 测试用例设计：根据需求文档设计详细的测试用例
- 测试数据准备：准备测试所需的模拟数据
- 测试执行：按照测试用例执行测试，记录测试结果
- 缺陷管理：记录和跟踪测试过程中发现的缺陷
- 测试报告：生成测试报告，总结测试结果和发现的问题

(3) 围绕被测系统，你如何设计单元测试内容，请对单元测试用例数量及覆盖率情况进行说明。

单元测试内容设计：

1. 用户管理模块：
 - 注册功能测试
 - 登录功能测试
 - 个人信息管理测试
 - 地址管理测试

2. 商品管理模块：
 - 商品列表测试
 - 商品详情测试
 - 商品分类测试
 - 评价系统测试
3. 订单管理模块：
 - 购物车测试
 - 订单创建测试
 - 订单状态管理测试
 - 支付测试
4. 商家管理模块：
 - 商品发布测试
 - 库存管理测试
 - 订单处理测试
5. 管理员管理模块：
 - 用户管理测试
 - 商品审核测试
 - 商家审核测试
 - 数据统计测试

单元测试用例数量及覆盖率：

- 测试用例数量：共设计了 200+ 个单元测试用例，覆盖系统的核心功能
- 代码覆盖率：代码覆盖率达到 85% 以上，包括：
 - 语句覆盖率：90%
 - 分支覆盖率：85%
 - 路径覆盖率：80%
- 测试重点：重点测试核心业务逻辑和关键功能，如用户认证、订单处理、支付流程等

(4) 能否举例说明在你的项目中，如何结合业务需求和测试目标进行测试用例设计？

举例：订单创建功能测试

业务需求：

- 用户可以将商品加入购物车，然后创建订单
- 订单创建需要验证用户身份、商品库存、价格等信息
- 订单创建成功后，需要更新商品库存和生成订单记录

测试目标：

- 验证订单创建功能是否正常
- 验证订单创建过程中的业务规则是否正确
- 验证订单创建后的数据一致性

测试用例设计：

1. 正常场景：
 - 测试用例 1：用户登录后，将商品加入购物车，创建订单，验证订单创建成功
 - 测试用例 2：用户登录后，直接购买商品，创建订单，验证订单创建成功
2. 异常场景：
 - 测试用例 3：未登录用户尝试创建订单，验证系统提示登录
 - 测试用例 4：商品库存不足时创建订单，验证系统提示库存不足
 - 测试用例 5：商品价格发生变化时创建订单，验证系统使用最新价格

- 测试用例 6: 用户地址信息不完整时创建订单, 验证系统提示完善地址

3. 边界场景 :

- 测试用例 7: 购买数量为 1 的商品, 验证订单创建成功
- 测试用例 8: 购买数量达到库存上限的商品, 验证订单创建成功
- 测试用例 9: 购买多个不同商品, 验证订单创建成功

(5) 在你的项目中, 你是如何实施回归测试的? 它对于保证软件质量有什么重要意义?

回归测试实施 :

1. 测试环境准备 : 搭建与生产环境相似的测试环境
2. 测试用例选择 : 选择与修改相关的测试用例, 以及核心功能的测试用例
3. 测试执行 : 按照测试用例执行回归测试, 记录测试结果
4. 缺陷管理 : 记录和跟踪测试过程中发现的缺陷
5. 测试报告 : 生成回归测试报告, 总结测试结果和发现的问题

重要意义 :

- 保证软件质量 : 确保修改不会引入新的问题, 保证系统的稳定性
- 提高开发效率 : 通过自动化回归测试, 减少手动测试的工作量
- 降低风险 : 及时发现和修复回归问题, 降低系统上线的风险
- 增强信心 : 通过回归测试, 增强对系统质量的信心
- 持续改进 : 通过回归测试, 不断发现和修复系统问题, 持续改进系统质量

(6) 你如何评估被测系统的性能需求的, 采用了什么测试方法及怎样实施的?

性能需求评估 :

1. 用户并发量 : 根据系统预期的用户量, 评估系统需要支持的并发用户数
2. 响应时间 : 评估系统在不同场景下的响应时间要求, 如商品列表加载时间、订单创建时间等
3. 吞吐量 : 评估系统在单位时间内能够处理的请求数
4. 资源使用 : 评估系统的 CPU、内存、磁盘等资源使用情况

测试方法 :

- 负载测试 : 测试系统在不同负载下的性能表现
- 压力测试 : 测试系统在极限负载下的性能表现
- ** endurance 测试 **: 测试系统在长时间运行下的性能表现
- 峰值测试 : 测试系统在峰值负载下的性能表现

实施过程 :

1. 测试环境准备 : 搭建与生产环境相似的测试环境
2. 测试工具配置 : 使用 Jmeter 配置测试场景和参数
3. 测试执行 : 按照测试计划执行性能测试, 记录测试结果
4. 数据分析 : 分析测试结果, 识别性能瓶颈
5. 优化建议 : 根据测试结果, 提出性能优化建议

(7) 在执行测试用例的过程中, 你遇到了哪些挑战? 你是如何解决的?

挑战及解决方案 :

1. 测试环境搭建 :
 - 挑战: 搭建与生产环境相似的测试环境比较困难
 - 解决方案: 使用 Docker 容器技术, 快速搭建测试环境, 确保环境一致性
2. 测试数据准备 :
 - 挑战: 准备大量的测试数据比较耗时

- 解决方案：编写数据生成脚本，自动生成测试数据，提高测试效率
- 3. 测试用例设计：
 - 挑战：设计全面的测试用例比较困难
 - 解决方案：参考需求文档和业务流程，设计详细的测试用例，确保测试覆盖度
- 4. 性能测试：
 - 挑战：模拟真实的用户行为和负载比较困难
 - 解决方案：使用 Jmeter 等工具，模拟真实的用户行为和负载，进行性能测试
- 5. 缺陷管理：
 - 挑战：跟踪和管理测试过程中发现的缺陷比较困难
 - 解决方案：使用缺陷管理工具，如 Jira，跟踪和管理缺陷，确保缺陷及时修复

(8) 你是如何进行缺陷管理的？包括缺陷的发现、记录、跟踪和验证等流程。

缺陷管理流程：

1. 缺陷发现：通过测试用例执行、用户反馈等方式发现缺陷
2. 缺陷记录：使用缺陷管理工具（如 Jira）记录缺陷，包括缺陷描述、复现步骤、预期结果、实际结果等信息
3. 缺陷分类：根据缺陷的严重程度和优先级进行分类
4. 缺陷分配：将缺陷分配给相应的开发人员进行修复
5. 缺陷跟踪：跟踪缺陷的修复状态，确保缺陷及时修复
6. 缺陷验证：缺陷修复后，进行验证测试，确保缺陷已修复
7. 缺陷关闭：验证通过后，关闭缺陷

缺陷管理工具：

- Jira：用于缺陷的记录、跟踪和管理
- Confluence：用于缺陷的文档管理
- Git：用于缺陷修复的代码管理

(9) 你在项目中是否使用了自动化测试？请分享你使用的自动化测试工具和框架。

自动化测试：

- 前端自动化测试：
 - 工具：Vitest、Playwright
 - 框架：Vue Test Utils
 - 测试内容：组件测试、页面测试、交互测试
- 后端自动化测试：
 - 工具：JUnit、Mockito
 - 框架：Spring Test
 - 测试内容：单元测试、集成测试、API 测试
- 性能自动化测试：
 - 工具：Jmeter
 - 测试内容：负载测试、压力测试、endurance 测试
- 安全自动化测试：
 - 工具：OWASP ZAP
 - 测试内容：漏洞扫描、渗透测试

(10) 你使用过哪些测试管理工具？它们在你的测试工作中起到了什么作用？

测试管理工具：

1. Jira：
 - 作用：用于缺陷管理、测试任务管理、测试进度跟踪

- 功能：缺陷记录、缺陷跟踪、缺陷分配、缺陷状态管理

2. Confluence :

- 作用：用于测试文档管理、测试计划管理
- 功能：测试计划编写、测试用例管理、测试报告生成

3. Zephyr :

- 作用：用于测试用例管理、测试执行管理
- 功能：测试用例设计、测试执行跟踪、测试覆盖率分析

4. TestRail :

- 作用：用于测试用例管理、测试执行管理
- 功能：测试用例设计、测试执行跟踪、测试报告生成

5. Jenkins :

- 作用：用于持续集成和自动化测试
- 功能：自动构建、自动测试、测试结果通知

(11) 在你的项目中，自动化测试覆盖了哪些测试类型？它如何提高了测试效率和质量？

自动化测试覆盖的测试类型：

1. 单元测试：对核心功能模块进行单元测试，确保功能正确性
2. 集成测试：测试不同模块之间的交互，确保系统整体功能正常
3. API 测试：测试后端 API 的功能和性能
4. UI 测试：测试前端界面的功能和交互
5. 性能测试：测试系统在高并发场景下的性能表现
6. 回归测试：测试修改是否引入新的问题

提高测试效率和质量：

- 提高测试效率：
 - 自动化测试可以快速执行大量的测试用例，减少手动测试的工作量
 - 自动化测试可以在每次代码提交后自动执行，及时发现问题
 - 自动化测试可以在不同环境下执行，确保环境一致性
- 提高测试质量：
 - 自动化测试可以避免人为错误，提高测试的准确性
 - 自动化测试可以覆盖更多的测试场景，提高测试的覆盖度
 - 自动化测试可以重复执行，确保测试的一致性

(12) 请基于你整个测试范围及测试的执行结果，对你的被测系统进行综合的评价。

综合评价：

- 功能完整性：系统实现了所有需求功能，包括用户管理、商品管理、订单管理、商家管理、管理员管理、租赁系统、二手交易等模块
- 性能表现：系统在正常负载下响应速度良好，满足用户需求
- 安全性：系统实现了基本的安全措施，如 JWT 认证、权限管理、数据加密等
- 可用性：系统界面友好，操作便捷，用户体验良好
- 可靠性：系统稳定运行，没有出现严重的故障或错误
- 可扩展性：系统架构设计合理，具有良好的可扩展性

测试结果：

- 功能测试：共执行 200+个测试用例，通过率 95%以上
- 性能测试：系统在 100 并发用户下响应时间小于 2 秒
- 安全测试：未发现严重的安全漏洞
- 兼容性测试：系统在主流浏览器和设备上表现良好

改进建议：

- 进一步优化系统性能，提高并发处理能力
- 增强系统安全性，防止各种安全攻击
- 提高测试覆盖度，确保系统质量
- 优化用户体验，提高系统的易用性

(13) 你对当前的软件测试前沿技术有哪些了解？比如 AI 在测试中的应用、测试驱动开发等。

软件测试前沿技术：

1. AI 在测试中的应用：

- 智能测试用例生成：使用 AI 自动生成测试用例，提高测试覆盖度
- 智能缺陷检测：使用 AI 自动检测代码中的缺陷，提高缺陷发现率
- 智能测试执行：使用 AI 优化测试执行策略，提高测试效率
- 智能测试分析：使用 AI 分析测试结果，识别系统的弱点和改进方向

2. 测试驱动开发（TDD）：

- 概念：先编写测试用例，然后编写代码实现功能，最后运行测试验证功能
- 优势：提高代码质量，减少缺陷，促进代码重构，提高开发效率
- 实践：在项目中部分采用 TDD 方法，编写测试用例驱动代码开发

3. 持续集成/持续测试（CI/CT）：

- 概念：每次代码提交后自动执行测试，确保代码质量
- 工具：Jenkins、GitLab CI、GitHub Actions 等
- 实践：在项目中使用 Jenkins 实现持续集成和持续测试

4. 自动化测试框架：

- 前端：Cypress、Playwright、Puppeteer 等
- 后端：JUnit 5、TestNG、Mockito 等
- API：Postman、RestAssured 等
- 性能：Gatling、Locust 等

5. DevOps：

- 概念：开发和运维一体化，实现快速交付和持续改进
- 实践：在项目中引入 DevOps 理念，实现开发、测试、部署的自动化

6. 混沌工程：

- 概念：通过注入故障来测试系统的韧性和可靠性
- 工具：Chaos Monkey、Gremlin 等
- 实践：在项目中进行简单的混沌测试，验证系统的容错能力

7. 无代码测试：

- 概念：使用可视化工具创建测试用例，无需编写代码
- 工具：Testim、TestCraft 等
- 优势：降低测试门槛，提高测试效率

8. 云测试：

- 概念：在云环境中执行测试，提供弹性和可扩展性
- 工具：AWS Device Farm、BrowserStack 等
- 优势：可以测试不同设备和环境，提高测试覆盖度